Spis treści

1. Wstęp 1

2. Cel pracy 2

3. Technologie 3

3.1. Mikroprocesor 3

3.2. Wyświetlacz 3

4. System czasu rzeczywistego 5

5. Implementacja 8

5.1. Enkoder 8

5.2. Przycisk 10

5.2. Obiekt 10

5.3. Regulator 12

5.4. Wyświetlacz 13

6. Podsumowanie 15

Bibliografia 16

1. Wstęp

System dedykowany jest systemem przeznaczonym do spełniania ściśle sprecyzowanego celu. Przykładowym systemem tego typu może być symulator obiektu automatyki wraz z regulatorem PID. Indywidualną potrzebą tak wykreowanego systemu byłoby zarządzanie teoretycznym obiektem, biorąc pod uwagę czynniki przewidziane przez twórcę. Wraz z implementacją obiektu konieczna byłaby implementacja regulatora. Najoczywistszym wyborem w tej kwestii byłby najbardziej popularny oraz uniwersalny regulator typu PID. Składający się z trzech członów, proporcjonalnego, całkującego oraz różniczkującego. Zastosowanie go umożliwiłoby poprawne sterowanie większością znanych obiektów. Tak zaprojektowany symulator wymaga jednak komunikacji z użytkownikiem, czyli osobą obsługującą go. Wymaganym elementem byłby zatem interfejs użytkownika oraz opracowany sposób wymiany danych. Bardzo przydatny w tej kwestii okazałby się wyświetlacz graficzny, ułatwiłby on komunikację mieszcząc wszystkie ważne parametry w jednym miejscu oraz przyczyniłby się do polepszenia warunków pracy z symulatorem, poprzez wizualizację pewnych istotnych części systemu.

2. Cel pracy

Celem pracy jest zaimplementowanie prostego obiektu wraz z regulatorem na platformie mikroprocesorowej, z wykorzystaniem systemu operacyjnego czasu rzeczywistego. Regulator będzie miał możliwość interaktywnej zmiany parametrów. W tym celu wykorzystany zostanie wyświetlacz, do obrazowania danych i nastaw, oraz zestaw przełączników, do zmiany wartości poszczególnych zmiennych. Przeanalizowana zostanie również możliwość wykorzystania wolnego elementu jakim jest wyświetlacz połączony komunikacją SPI, do wyświetlania danych w czasie rzeczywistym, takich jak charakterystyki oraz wpływ jego zastosowania na możliwości regulacji.

Czas	Zadanie
1	Dobór materiałów, utworzenie planu projektu, cele i metody.
2	Implementacja wyświetlacza, enkodera oraz menu, wstęp do regulatora i obiektu.
3	Wyświetlanie parametrów, implementacja regulatora i obiektu.
4	Implementacja wyświetlacza - praca z charakterystykami / laptop do wyświetlania danych.

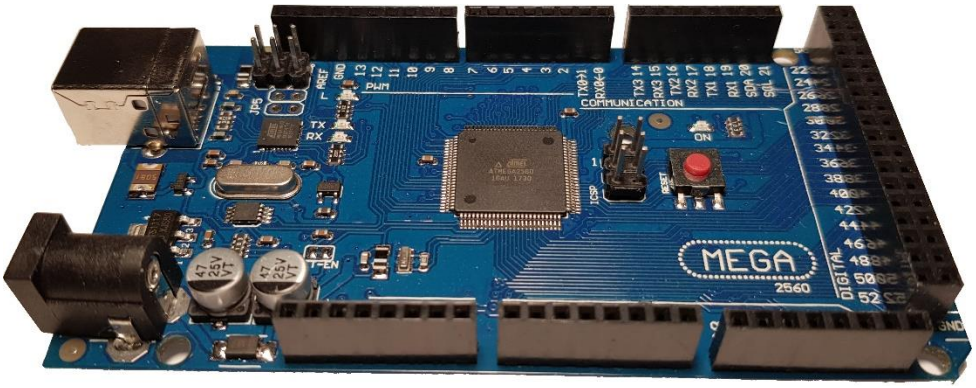
Tabela 1 Harmonogram

3. Technologie

W rozdziale tym przedstawione zostaną możliwe do wykorzystania w projekcie technologie. Podstawowymi narzędziami będą mikrokontroler będący bazą wraz z wyświetlaczem odpowiedzialnym za obsługę komunikacji pomiędzy urządzeniem a człowiekiem. W tym celu wykorzystane zostaną również elementy służące do odbierania informacji od osoby obsługującej regulator.

3.1. Mikroprocesor

W celu realizacji poczynionych założeń wymagane jest wykorzystanie mikrokontrolera. Rynek oferuje różnorodność tego typu rozwiązań, począwszy od samoistnych układów mikroprocesorowych, aż po zabudowane płytki zawierające w sobie również programator i układy zasilające. Druga możliwość bardzo dobrze nadaje się do tworzenia prototypów oraz bardzo często oferuje darmowe środowisko programistyczne. Tego typu platformą jest Arduino. W projekcie zostanie wykorzystane Arduino Mega (1).



Rysunek 1 Arduino Mega

Wyposażona jest w pięćdziesiąt cztery cyfrowe piny, z czego piętnaście z nich może pełnić funkcję PWM. Posiada również szesnaście wejść analogowych. Pracuje na logice pięcio-woltowej oraz obsługuje komunikację SPI oraz szeregową. Zatem spełnia wszystkie podstawowe założenia odnośnie projektu.

3.2. Wyświetlacz

Za obsługę komunikacji pomiędzy użytkownikiem, a urządzeniem odpowiadał będzie wyświetlacz graficzny LCD 3.2”, obsługujący komunikację SPI. Jego podstawowym celem będzie wyświetlanie parametrów regulatora oraz zakłóceń czy też uchybu regulacji i wartości rzeczywistej.

Rysunek 2 Wyświetlacz LCD (2)

Ciekawym rozwiązaniem, które zostanie przeanalizowane będzie możliwość wyświetlania charakterystyki czasowej regulacji, wykorzystując dane rzeczywiste. Rozważone będą dwie możliwości. Pierwsza będzie określała szybkość zmian z jaką będzie mógł zachodzić proces regulacji, aby charakterystyki mogły być rysowane w czasie rzeczywistym. Konieczne będzie w tym sposobie zapewnienie aktualności danych, a zatem szybkość regulacji byłaby ograniczona przez szybkość wyświetlacza. Druga będzie oferowała możliwość regulacji nie zważając na szybkość odświeżania wyświetlacza. W tym jednak sposobie regulacja przebiegać będzie w czasie rzeczywistym, a wykresy będą rysowane na podstawie danych historycznych, w przypadku, gdy cały proces regulacji będzie szybszy niż rysowanie charakterystyk. Rozwiązaniem tego problemu mogłoby być rysowanie tylko co którejś próbki sygnału, w przypadku wolnych procesów problem by nie występował. Problemem byłoby jednak magazynowanie danych, które w bardzo szybkich procesach byłyby generowane w bardzo dużych ilościach. Założyć jednak można, iż tego typu procesy nie będą obsługiwane. Idąc jednak dalej tym tokiem rozumowania, można założyć, że obsługiwane będą procesy na tyle wolne, aby można było zaimplementować opcję pierwszą, która mimo, iż ogranicza możliwości regulatora wydaję się najlepszą. Są to jednak rozważania, których rozwiązanie nastąpi dopiero podczas implementacji projektu. W tym momencie przemyślany zostanie również mechanizm adaptacji skali do wyświetlania.

Całość wprowadzania parametrów będzie obsługiwana poprzez enkoder wraz z przyciskiem monostabilnym. Przycisk będzie służył do zatwierdzania oraz przejścia do kolejnego parametru urządzenia. Enkoderem zaś ustawiane będą wartości.

4. System czasu rzeczywistego

Wykorzystanym systemem czasu rzeczywistego jest freeRTOS (3; 4; 5). Jego implementacja oraz wykorzystanie jest stosunkowo dobrze opisane przy dokumentacji do biblioteki Arduino. Biblioteka dołączana do programu pozwalająca wykorzystać system wyposażona jest w przykłady oraz opis funkcji. Podczas pracy nad projektem skorzystano z wymiany danych pomiędzy zadaniami, za pomocą kolejek oraz semaforów. Przykład utworzenia kolejki pokazano na [Rysunek 3].

```
QueueHandle_t regulatorQueue;
.
.
.
regulatorQueue = xQueueCreate(16, sizeof(parametr));
if(regulatorQueue == NULL)                                  //Sprawdzenie czu utworzono kolejkę
{
  Serial.println("regulatorQueue nie może zostać utworzona");    //Komunikat o błedzie
  while(true);
}
```

Rysunek 3 Kolejka regulatora

Początkowym etapem jest zdefiniowanie kolejki, a następnie utworzenie jej funkcją *xQueueCreate()*, przyjmującej dwa argumenty. Pierwszym z nich jest wielkość kolejki, podana w elementach. Drugim jest wielkość pojedynczego elementu. Następnie sprawdzane jest czy kolejka została utworzona, jeśli nie, wyświetlany jest komunikat.

```
xQueueSend(regulatorQueue, &parametry, portMAX_DELAY);
```

Rysunek 4 Dodawanie elementu do kolejki

Dodawanie elementu do kolejki [Rysunek 4] odbywa się poprzez podanie uchwytu do kolejki, którą chcemy użyć, następnie podawany jest wskaźnik na dodawany element oraz ustawiany jest czas w postaci ilości ticków, na które zostanie zablokowane zadanie, w oczekiwaniu na miejsce w kolejce.

```
if( xQueueReceive( regulatorQueue, &( parametry ),( TickType_t ) 1 ) == pdPASS )
```

Rysunek 5 Odbiór danych z kolejki

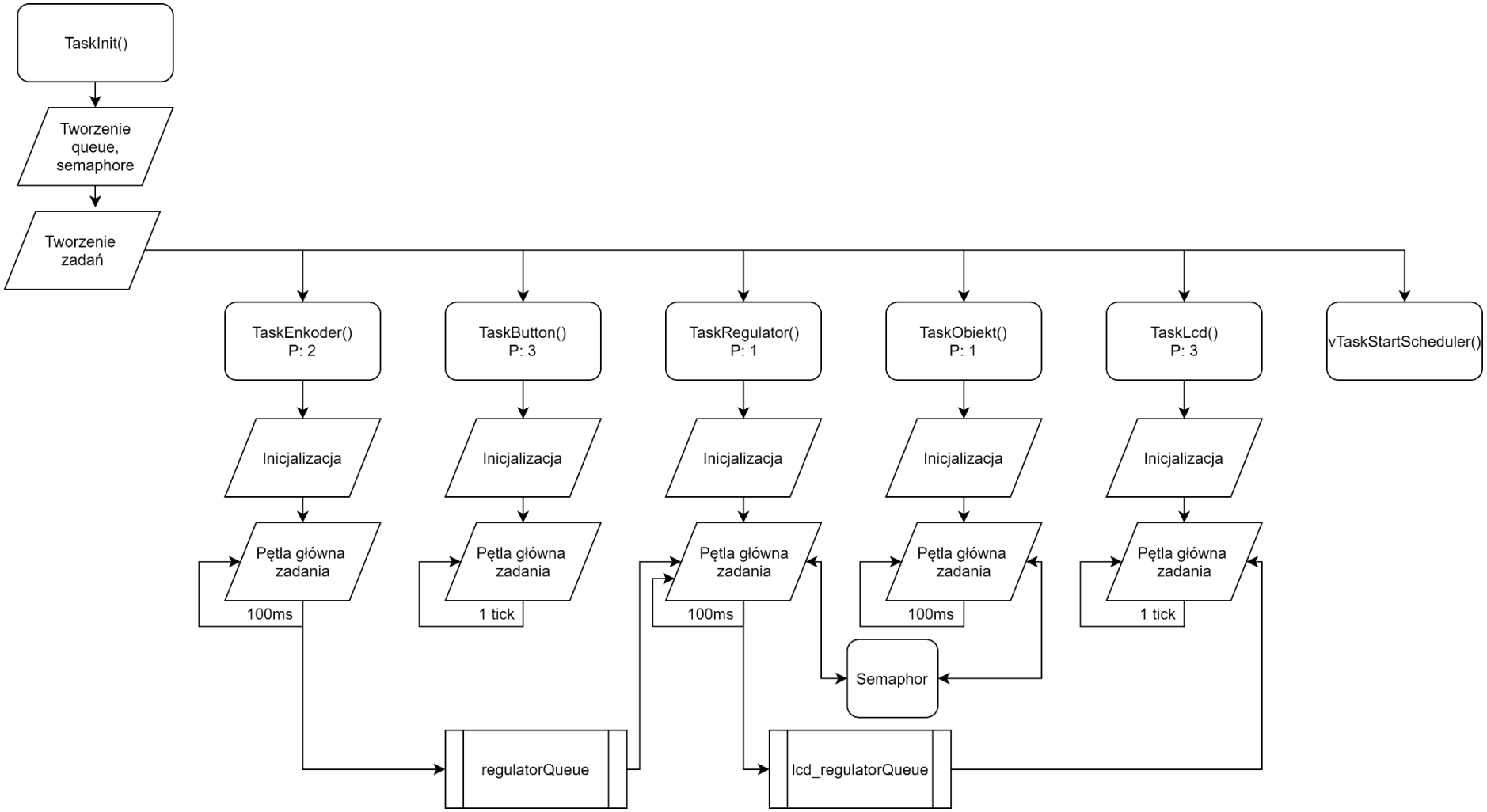
Bardzo podobnie wygląda sytuacja przy odbiorze danych z kolejki.

Kolejnym elementem system wykorzystywanym do wymiany danych jest semafor. Tworzy się go podobnie do kolejek, lecz nie przyjmuje żadnych parametrów. Wywołanie go następuje korzystając z funkcji [Rysunek 6].

```
xSemaphoreTake( serial_mutex, portMAX_DELAY );
xSemaphoreGive( serial_mutex );
```

Rysunek 6 Użycie semaforów

Pomiędzy wywołanymi funkcjami znajdują się współdzielone dane, na których odbywa się praca.



Rysunek 7 Ogólny schemat programu

5. Implementacja

W rozdziale tym przedstawiona zostanie praktyczna realizacja postawionych problemów, pokazane oraz omówione zostaną wyodrębnione fragmenty kodu.

5.1. Enkoder

Enkoder wykorzystany w projekcie ma za zadanie realizować wprowadzanie danych do regulatora. Wraz z przyciskiem monostabilnym tworzy menu. Przycisk służy do wyboru zmienianego parametru, enkoderem zaś dany parametr się zmienia. Początkowym etapem implementacji tego rozwiązania jest zdefiniowanie zmiennych.

```
unsigned long time = 0;              //czas
volatile long pozycjaEnkodera = 0;   //Pozycja biezaca enkodera
#define pinA         2               //Pin A - przerwanie
#define pinB         3               //Pin B
```

Rysunek 8 Definicje zmiennych enkodera

Definiowana jest zmienna czasu, za pomocą której możemy programowo zwolnić odczytywanie danych z enkodera. Określane zostają używane piny, z czego 2. pozwala na obsługę przerwań w Arduino Uno. Dwie pozostałe zmienne służą do odczytów wartości, z enkodera. Następnie utworzona zostaje funkcja *isr()*, obsługująca przerwanie.

```
pinMode(pinA, INPUT);                                    //Zainicjowanie pinu enkodera A
pinMode(pinB, INPUT);                                    //Zainicjowanie pinu enkodera B
attachInterrupt(digitalPinToInterrupt(pinA), isr, RISING);  //Utworzenie obsługi przerwania
time = millis();                                          //Zapisanie czasu w zmiennej
```

Rysunek 9 Początkowe ustawienia

Odpowiednim pinom zostaje przypisana ich rola, zapisywany zostaje również czas początkowy. Tworzona jest także obsługa przerwania. Gdy na pinie drugim, podłączonym do odpowiedniej nóżki enkodera zmieni się stan z niskiego na wysoki następuje wywołanie funkcji *isr()*. Odczytywane są w niej wartości obu pinów oraz sprawdzane jest czy czas odstępu pomiędzy zmianą parametrów jest zachowany. Kolejnym krokiem jest użycie instrukcji warunkowej *if else* oraz zwiększenie lub zmniejszenie zmiennej odpowiadającej za pozycję enkodera. Na końcu zapisywany jest czas do zmiennej *time*. W ten właśnie sposób realizowany jest odczyt z enkodera. Kolejnym krokiem do utworzenia menu zmiany parametrów jest przetworzenie tej wartości. Odbywa się to w zadaniu *TaskEnkoder()* tworzonym w pierwszej wywoływanej funkcji, przygotowującej mikrokontroler do pracy.

```
xTaskCreate(TaskEnkoder, "ZadanieEnkodera", 512,  NULL, 2, NULL);   //Zadanie enkodera
```

Rysunek 10 Zadanie enkodera

Zadanie przyjmuje sześć parametrów, z czego pierwszy określa wskaźnik na funkcję obsługującą to zadanie. Drugi jest nazwą widoczną tylko dla programisty. Trzeci parametr to wielkość stosu wyrażona w słowach, dobrana eksperymentalnie. Przy stosie 128 zadanie nie wykonywało się poprawnie, dlatego wartość ta została zwiększona. Trzy ostatnie parametry to wskaźnik do przekazywanych do funkcji parametrów, priorytet oraz nieużywany przez nas „uchwyt". Obecnie program obsługuje dwa zadania, enkodera oraz przycisku. Po ich otworzeniu uruchamiany jest *vTaskStartScheduler();* do zarządzania zadaniami.

Zadanie enkodera w początkowej fazie wykonywania inicjalizuje wyświetlacz oraz generuje na nim podstawowe informacje. W tym momencie są to elementy zaznaczone na czerwono na rysunku [Rysunek 11].

Rysunek 11 Menu

Aktualnie zmieniany parametr podkreślony jest czerwoną linią, po naciśnięciu przycisku przechodzi się do kolejnego parametru. Po ustawieniu wszystkich następuje wysłanie danych do

zadania obsługującego regulator. Całość zrealizowana jest na zasadzie *switch case*, w zależności od poziomu menu.

```
case 0:                                          //Zmiana zmiennej Kp
parametry.KP = (pozycjaEnkoderaOld/10);
if (parametry.KP<=0)                                   //Ograniczenia zmiennej
{
  parametry.KP = 0;
}
wyswietlanie(40, 1, parametry.KP);                 //Wyświetlanie zmiennej
break;
```

Rysunek 12 Zmiana parametru Kp

W danym *case* pozycja z enkodera przepisywana jest na zmienną lokalną odpowiadającą danemu parametrowi. Następnie sprawdzane są ograniczenia danego parametru oraz wywoływania jest funkcja wyświetlająca zmienne na wyświetlaczu.

```
case 4:                                                  //Wysyłanie parametrów
tft.drawLine(0, 38, tft.width(),38, ILI9341_BLACK);      //Czerni starą linię
xQueueSend(regulatorQueue, &parametry, portMAX_DELAY);   //Wysyła parametry do regulatora
poziomMenu = 5;
break;
```

Rysunek 13 Wysyłanie do zadania regulatora

W przypadku, gdy wszystkie parametry zostaną ustawione wysyłana zostanie zmienna typu struct zawierająca dane.

5.2. Przycisk

Funkcję przejścia pomiędzy zmianami danych parametrów realizuje przycisk monostabilny. Obsługuje go osobne zadanie, sprawdzające stan pinu do którego element jest podłączony. Początkowo tworzone są zmienne stanu bieżącego oraz starego stanu przycisku, a następnie zmienne czasu trwania danego stanu. Jeśli na pinie pojawi się stan wysoki, sprawdzane jest czy poprzedni stan był stanem niskim. Następnie sprawdzana jest różnica pomiędzy czasami, w którym wykryto stan wysoki oraz aktualnym. Jeśli różnica ta jest większa niż 100ms zmieniany jest poziom menu. Różnica czasów może zostać ustawiona na większą, tak aby przy zmianie kolejnych stopni menu wymagane było przytrzymanie przycisku przez dłuższy czas.

5.2. Obiekt

Elementem potrzebnym do symulacji sterowania jest obiekt regulacji. Zaimplementowano prosty obiekt (6). Całość obliczeń, tak jak w przypadku regulatora odbywa się również z

użyciem semafora. W pętli obliczana jest wartość bieżąca obiektu, również z ograniczeniami. Zadanie to również wykonuje się co 100ms.

Zbadano dwa obiekty o następujących transmitancjach:

$$G_1(s) = \frac{K}{(1+Ts)} \text{ oraz } G_2(s) = \frac{K_1 \times K_2}{(1+T_1 s)\times(1+T_2 s)}$$

Co w implementacji wyglądało następująco, dla G_1:

$$h_{(n+1)} = e^{\left(-\frac{t_i}{T}\right)} \times h_n + K * \left[1 - e^{\left(-\frac{t_i}{T}\right)}\right] * Q_n$$

, gdzie:

$$Q\ - Sygnał\ sterujący$$

$$h\ - poziom\ cieczy\ w\ zbiorniku$$

Bardzo podobnie wyglądało to również dla G_2:

$$h_{1_{(n+1)}} = e^{\left(-\frac{t_i}{T_1}\right)} \times h_{1_n} + K_1 * \left[1 - e^{\left(-\frac{t_i}{T_1}\right)}\right] * Q_n$$

$$h_{2_{(n+1)}} = e^{\left(-\frac{t_i}{T_2}\right)} \times h_{2_n} + K_2 * \left[1 - e^{\left(-\frac{t_i}{T_2}\right)}\right] * h_{1_n}$$

, gdzie:

$$Q\ - Sygnał\ sterujący$$

$$h_{1,2}\ - poziom\ cieczy\ w\ zbiornikach$$

Ostatecznie ten obiekt został zaimplementowany.

```
H1 = exp(-h/T1)*h1+K1*(1-exp(-h/T1))*sygnalSterujacy;
H2 = exp(-h/T2)*h2+K2*(1-exp(-h/T2))*h1;
h1 = H1;
h2 = H2;
wartoscBiezaca = H2;
```

Rysunek 14 Implementacja obiektu

Wszelkie wartości z obiektu oraz regulatora wyświetlane są na wyświetlaczu LCD [Rysunek 15].

Rysunek 15 Wyświetlacz LCD - wartości obiektu

W górnej części ekranu są przedstawione parametry regulatora, wzmocnienie, czas całkowania, różniczkowania oraz wartość zadana. Wartość zadana wyrażona jest w procentach, czasy w sekundach. W dolnej części po lewej stronie wypisane są najważniejsze wartości procesu regulacji. E oznacza uchyb, U to wartość sygnału sterującego, a Y jest wartością bieżącą obiektu. Po prawej stronie przedstawiono teoretyczny obiekt regulacji, jako wypełnienie pewnego zbiornika. Czerwoną kreską oznaczono wartość zadaną, do której dąży obiekt. Niebieski kolor to poziom jego wypełnienia, czyli poziom wartości bieżącej, również wyskalowanej w procentach. Uchyb to różnica pomiędzy czerwoną kreską, a niebieskim polem.

5.3. Regulator

Utworzone zostało osobne zadanie regulatora, zawierające w sobie cyfrową implementacje regulatora PID w zamkniętej pętli sprzężenia zwrotnego. Początkowym etapem zadania jest odebranie parametrów regulatora. Takich jak wzmocnienie Kp, czas całkowania, czas różniczkowania oraz wartość zadaną. Następnie na dalszą część programu wywoływana jest funkcja semafora, gdyż zadanie korzysta z współdzielonych zmiennych z zadaniem symulującym obiekt.

```
xSemaphoreTake( serial_mutex, portMAX_DELAY );
```

Rysunek 16 Wywołanie funkcji semafora

W kolejnych krokach obliczany jest uchyb oraz sygnał sterujący. Uchyb zapisany jest jako różnica wartości zadanej i wartości bieżącej. Sygnał sterujący to suma wszystkich składowych poszczególnych członów. Następnie zaimplementowany jest prosty system anti windup.

```
if (sygnalSterujacy<=0) { sygnalSterujacy=0;  suma=uchyb;}
if (sygnalSterujacy>umax) {sygnalSterujacy=umax; suma-=uchyb;}
```

Rysunek 17 Anti windup

W kolejnym kroku wysyłana jest kolejka do zadania obsługującego wyświetlanie, oraz zdejmowany jest semafor. Przyjęto, iż zadania będą wykonywały się co 100ms, dzięki temu można było przyjąć sztywny okres impulsowania jako 0.1 część sekundy.

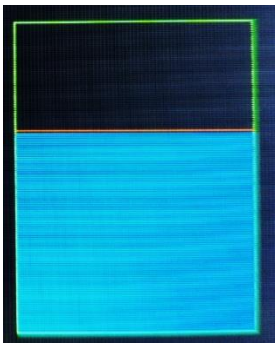
5.4. Wyświetlacz

Komunikację pomiędzy użytkownikiem, a urządzeniem zapewnia wyświetlacz wraz z zestawem złożonym z enkodera i przycisku.

```
tft.drawRect(170, 60, 130, 170, ILI9341_GREEN);     //Rysuje zbiornik
// //Zbiornik wysoki na 168
//  ___________
// |           | 168
// |           |
// |           |
// |           |
// |           |
// |           |
// |           |
// |___________| 0
poziom = abs(round((lcdObiekt.wartoscBiezaca/100)*168));
tft.fillRect(171, 61, 128, 168-poziom, ILI9341_BLACK);
tft.fillRect(171, 229-poziom, 128, poziom, ILI9341_BLUE);
tft.drawLine(171, 229-round((lcdObiekt.WZ/100)*168), 299, 229-round((lcdObiekt.WZ/100)*168), ILI9341_RED);
```

Rysunek 18 Rysowanie zbiornika

Korzystając z biblioteki dostarczonej przez producenta podobnego elementu (7) zaimplementowano symulację wypełnienia zbiornika, czyli symulowanego obiektu. Początkowym etapem wyrysowywania obrazu jest szkic zbiornika w kolorze zielonym. Następnie przeliczony zostaje stopień wypełnienia zbiornika, tak aby dostosować go do zaplanowanych stu sześćdziesięciu ośmiu pikseli ekranu. Następnie wypełniane są dwa prostokąty, jeden czernią, obrazujący pustą część zbiornika. Drugi wypełniany jest niebieskim kolorem, obrazując poziom cieczy. Ostatnim etapem jest zaktualizowanie położenia linii wyznaczającej wartość zadaną, oznaczoną czerwonym kolorem.



Rysunek 19 Zbiornik na wyświetlaczu

Co w efekcie końcowym daje efekt napełniania się zbiornika [Rysunek 19]. Czerwona linia wartości zadanej bardzo dobrze kontrastuje z resztą kolorów użytych przy rysowaniu zbiornika.

6. Podsumowanie

Celem pracy było zasymulowanie obiektu wraz z regulatorem, wykorzystując w tym celu mikrokontroler wraz z interfejsem użytkownika. Wykorzystano enkoder wraz z przyciskiem do wprowadzania danych. Komunikację z użytkownikiem obsługuje wyświetlacz LCD 3.2”. Pokazane są na nim wszelkie zmieniane parametry regulatora oraz najważniejsze parametry obiektu, czy też regulacji. Dodatkowym graficznym udogodnieniem jest przedstawienie symulacji procesu za pomocą zbiornika wraz z animacją wypełniania go, w zależności od jego pracy. Wprowadzanie danych do regulatora działa sprawnie, jedynym mankamentem jest brak zachowywania starej wartości podczas wprowadzania nowej, co skutkuje zwiększeniem ilości obrotów enkodera, w celu ustawienia nowego parametru. Oznacza to, że podczas przełączania stopni menu, wartość nowy regulowany parametr przyjmuje wartość początkową poprzedniego regulowanego parametru, a nie rozpoczynamy od jego starej wartości. W tym kierunku na pewno powinna się zwrócić kolejna modyfikacja, ulepszenie projektu. Po ustawieniu wszystkich danych regulatora następuje proces regulacji, od bieżącego momentu. Oznacza to, iż obiekt zachowuje jako warunki początkowe swój poprzedni stan. Nie dodano możliwości resetu i natychmiastowego opróżnienia zbiornika. Kwestia wyświetlacza oraz jego wpływu na pracę symulatora wypadła pozytywnie, nie widać szczególnie długiego czasu ładowania elementów, co sprzyja poprawnemu odbiorowi oraz współpracy z urządzeniem.

Bibliografia

1. Arduino. [Online] [Zacytowano: 28 Kwiecień 2020.] https://store.arduino.cc/arduino-mega-2560-rev3.

2. Ili9341 LCD. [Online] [Zacytowano: 28 Kwiecień 2020.] https://pl.aliexpress.com/item/4000103273808.html?src=google&src=google&albch=shopping&acnt=494-037-6276&isdl=y&slnk=&plac=&mtctp=&albbt=Google_7_shopping&aff_platform=google&aff_short_key=UneMJZVf&&albagn=888888&albcp=6459787116&albag=77066918093&trgt=74.

3. Kamil. [Online] 8 Październik 2018. [Zacytowano: 28 Kwiecień 2020.] https://starter-kit.nettigo.pl/2018/10/system-operacyjny-czasu-rzeczywistego-dla-arduino/.

4. freeRTOS. [Online] [Zacytowano: 28 Kwiecień 2020.] https://www.freertos.org/a00106.html.

5. Łaguna Łukasz. [Online] 24 Styczeń 2019. [Zacytowano: 28 Kwiecień 2020.] http://www.embeddeddev.pl/kurs-freertos-semafory/.

6. *WIRTUALNY MODEL ZBIORNIKÓW Z WODĄ W STEROWNIKU PLC.* ZAWALICH Jacek. Nr 38, Gdańsk : Politechnika Gdańska, Wydział Elektrotechniki i Automatyki, Tom Zeszyty Naukowe Wydziału Elektrotechniki i Automatyki Politechniki Gdańskiej. Nr 36.

7. Adafruit. [Online] [Zacytowano: 28 Kwiecień 2020.] https://github.com/adafruit/Adafruit_ILI9341.